

Unity ECS 기반
대규모 3D 인스턴스 렌더링 성능 분석 및 최적화
중간보고서



팀명: 미오

분과: B

202255658 김재식 rla0578@pusan.ac.kr

202155593 이천서 angel5337@pusan.ac.kr

202255568 안지홍 anjhu@pusan.ac.kr

지도교수: 박영진

제출일: 2026년 7월 16일

부산대학교

목차

1	요구조건 및 제약사항 분석에 대한 수정사항	2
1.1	주요 수정사항	2
1.1.1	비교 조건을 렌더링 경로와 제출 방식의 조합으로 변경	2
1.1.2	조건별 공통 입력과 렌더링 설정을 통일	3
1.1.3	비교 요인 외의 렌더링 기능과 구현 범위를 제한	3
1.1.4	부하 수준과 반복 실행 조건을 현재 비교 구조에 맞게 조정	3
1.1.5	성능 계측과 결과 저장 요구사항을 구체화	4
2	설계 상세화 및 변경 내역	5
2.1	시스템 구조와 실험 조건	5
2.2	구현 구조	6
2.2.1	RMI 구현	7
2.2.2	BRG 구현	7
2.3	측정 및 실험 프로토콜	8
2.4	기대 결과 및 해석	10
2.5	착수보고서 대비 변경점	11
3	추진 계획 및 진행 현황	12
3.1	구성원별 진척도	14
3.2	구성원별 진척도	14
3.2.1	김재식: ECS 시뮬레이션 및 렌더링 구조	14
3.2.2	이천서: 군집 애니메이션 및 인스턴스 유형 구성	14
3.2.3	안지홍: 휴머노이드 렌더링과 성능 측정 자동화 및 실험 분석	15
3.3	위험 대응	15
4	중간 결과	16
4.1	구현 데모	16

4.2 실험、측정 도구	17
참고 문헌 및 자료	19

개요

실시간 대규모 3D 인스턴스 장면에서는 객체 수가 증가할수록 엔티티 컴포넌트 시스템(Entity Component System, ECS) 상태 갱신, 인스턴스 데이터 구성, 렌더링 명령 제출 및 GPU 처리에 필요한 부하가 함께 증가한다. 본 프로젝트는 Unity ECS 기반 군집 시뮬레이션과 GPU 스키닝 구조를 공통 기반으로 유지하면서, `Graphics.RenderMeshIndirect` 메서드(RMI)와 `BatchRendererGroup`(BRG)의 두 렌더링 경로에 통합 제출과 4분할 제출을 적용할 수 있는 대규모 인스턴스 렌더링 프로토타입과 성능 측정 도구를 구현하는 것을 목표로 한다. 착수보고서 이후 비교 대상과 실험 조건은 변경되었으나, ECS 시뮬레이션, 인스턴스 버퍼, GPU 스키닝 및 BRG 렌더러 등 기존 구현 기반은 유지하고 RMI 렌더러와 반복 측정 기능을 추가하는 방향으로 프로젝트를 조정하였다. 본 중간보고서는 착수보고서 이후 변경된 요구조건과 시스템 설계, 구현 진행 현황, 측정 도구 및 현재까지의 중간 산출물을 정리한다.

제1장 요구조건 및 제약사항 분석에 대한 수정사항

착수보고서 이후 프로토타입 구현과 비교 구조 검토를 진행하면서 비교 조건, 렌더링 구조, 실험 통제 및 성능 측정에 관한 요구조건을 구체화하였다. Unity ECS를 이용하여 대규모 3D 인스턴스의 상태를 갱신하고, GPU 버퍼 기반 렌더링 경로의 성능을 측정하여 구현 및 최적화 방향을 제시한다는 프로젝트의 상위 목표는 유지하였다.

다만 착수보고서에서 계획한 렌더 전용 Render-only BRG와 ECS 연계BRG(ECS-fed BRG)의 비교는 동등한 실행 조건을 구성하기 어렵다는 문제가 확인되었다. Render-only BRG는 ECS에서 갱신되는 이동, 대열, 교전 및 애니메이션 상태를 실제 렌더링 결과에 반영하지 않으므로 ECS-fed BRG와 동일한 기능을 수행하는 비교군으로 보기 어렵다. 또한 ECS 시뮬레이션 처리 여부와 렌더링 데이터 갱신량이 조건별로 달라 전체 부하가 일치하지 않으므로, 수집된 성능 차이가 데이터 공급 구조와 렌더링 구조 중 어느 요인에서 발생했는지 명확하게 해석하기 어렵고 측정 자료의 활용 범위도 제한된다.

이에 따라 모든 조건에서 동일한 ECS 군집 시뮬레이션을 실행하고, 해당 결과를 서로 다른 GPU 렌더링 경로로 전달하는 방식으로 비교 구조를 수정하였다. 현재 비교 대상은 RMI와 BRG이며, 각 경로에 통합 제출과 4분할 제출을 적용한다. 이러한 변경에 맞추어 공통 입력, 렌더링 설정, 구현 범위, 부하 수준, 반복 실행 및 성능 측정 요구조건을 다음과 같이 조정하였다.

1.1 주요 수정사항

1.1.1 비교 조건을 렌더링 경로와 제출 방식의 조합으로 변경

Render-only BRG와 ECS-fed BRG는 ECS 시뮬레이션 상태의 반영 여부와 처리 부하가 서로 달라 동등한 비교 조건으로 사용하기 어렵다. 따라서 데이터 공급 여부를 비교 요인에서 제외하고, 모든 조건에서 동일한 ECS 시뮬레이션 결과를 사용하도록 수정하였다.

수정된 비교 구조에서는 RMI와 BRG를 각각 통합 제출과 4분할 제출로 구현한다. E1은 RMI 통합 제출, E2는 RMI 4분할 제출, E3은 BRG 통합 제출, E4는 BRG 4분할 제출로 구성한다(표 2.1). 네 조건은 별도의 실행 썬 또는 실행 모드로 구성하되, 동일한 ECS 상태와

렌더링 자원을 입력으로 사용한다. 이를 통해 ECS 시뮬레이션 부하는 동일하게 유지하면서 렌더링 경로와 제출 구조의 차이를 비교할 수 있도록 한다.

1.1.2 조건별 공통 입력과 렌더링 설정을 통일

RMI와 BRG는 명령 제출 방식과 GPU 인스턴스 데이터 접근 구조가 다르기 때문에 동일한 GPU 버퍼, 셰이더 및 머티리얼 구현을 그대로 공유할 수 없다. 이러한 경로별 구현 차이가 시뮬레이션 상태나 화면 출력의 차이로 이어질 경우 렌더링 경로 자체의 성능을 비교하기 어렵다.

이에 따라 모든 조건에서 동일한 ECS 이동, 대열, 탐지, 전투 및 애니메이션 갱신 구조를 사용하고, 총 인스턴스 수, 스왑 구성, 초기 배치, 난수 시드, 카메라와 실행 시간을 동일하게 유지한다. 위치, 방향, 팩션 식별값과 애니메이션 상태는 동일한 ECS 원천 데이터에서 읽으며, 각 렌더러는 이를 경로별 GPU 데이터 구조로 변환한다.

경로별 셰이더와 머티리얼은 별도로 구성하되 메시, 텍스처, 애니메이션 및 GPU 스키닝 데이터는 공통으로 사용한다. 팩션별 색상과 조명 반응을 포함한 시각 파라미터와 렌더 상태도 동등하게 설정하여 두 렌더링 경로의 화면 출력이 동일하게 유지되도록 한다.

1.1.3 비교 요인 외의 렌더링 기능과 구현 범위를 제한

LOD, Shadow, Motion Vector, 개별 인스턴스 컬링 및 GameObject 기반 투사체 시각화가 활성화되면 렌더링 경로와 제출 분할 이외의 처리 비용이 측정 결과에 포함될 수 있다. 또한 GameObject 또는 개별 객체 단위 렌더링을 별도의 비교 조건으로 추가하면 현재 프로젝트의 구현 범위와 측정 조건이 과도하게 확대된다.

따라서 LOD는 0으로 고정하고 Shadow와 Motion Vector를 비활성화한다. 개별 인스턴스에 대한 프러스텀 및 오클루전 컬링은 수행하지 않으며, 모든 조건에서 동일한 인스턴스 집합을 가시 대상으로 제출한다.

원거리 공격 상태와 공격 애니메이션은 ECS 시뮬레이션에 유지하되, 비교에 직접 필요하지 않은 GameObject 기반 투사체 생성과 시각화는 측정 조건에서 비활성화한다. GameObject 또는 per-object draw 방식도 별도의 비교 조건으로 구현하지 않고, ECS 상태를 GPU 버퍼 기반 렌더링 경로로 전달하는 기능에 개발 범위를 집중한다.

1.1.4 부하 수준과 반복 실행 조건을 현재 비교 구조에 맞게 조정

착수보고서에서 설정한 인스턴스 규모와 반복 횟수는 변경된 E1-E4 비교 구조 및 현재 스왑 구성과 일치하지 않는다. 또한 제출 범위를 4개 구간으로 분할할 때 조건별 인스턴스 범위가 불균등하면 분할 제출 자체의 영향을 일관되게 비교하기 어렵다.

이에 따라 총 인스턴스 수 N 은 {2,000, 4,000, 8,000, 16,000}, 스왑 분할 수준 L 은 {4, 8, 16}으로 설정한다. 총 12개 부하 조합에서 E1-E4 조건을 각각 5회 반복하여 총 240회를 실행한다.

모든 N 은 L 과 제출 분할 수 4로 나누어떨어지도록 설정한다. 이를 통해 스왑별 인스턴스 수 N/L 과 4분할 제출 조건의 명령별 인스턴스 범위를 균등하게 유지한다. 각 실행은 초기화 이후 3초의 워밍업과 60초의 측정 구간으로 구성하며, 워밍업 구간에서 수집된 값은 최종 결과에 포함하지 않는다.

총 240회의 반복 실행을 수작업으로 수행할 경우 조건 설정 오류, 반복 회차 누락, 실행 시간 불일치 및 결과 파일 관리 오류가 발생할 수 있으므로 조건 선택, 초기화, 워밍업, 측정, 반복 회차 관리 및 CSV 저장을 자동화한다. 실행 조건과 검증 정보도 결과 파일에 함께 기록하여 잘못 설정되거나 누락된 실행을 식별할 수 있도록 한다.

1.1.5 성능 계측과 결과 저장 요구사항을 구체화

FPS와 Draw Command 중심의 기존 측정 항목만으로는 Main Thread, Render Thread 및 GPU 단계에서 발생하는 처리 차이를 구분하기 어렵다. 또한 RMI의 API 호출 및 간접 명령과 BRG의 BatchDrawCommand는 서로 다른 처리 계층의 값이므로 하나의 공통 명령 수로 직접 대응시키기 어렵다.

실제 경과시간 기반 Frame Time과 완료 프레임 수는 매 프레임 수집한다. Main Thread Time, Render Thread Time, GPU Time, 논리적 제출 명령 수, 계산상 업로드 바이트 및 경로별 구조 지표는 실제 경과시간 기준 1초 주기로 수집한다.

RMI 경로에서는 API 호출 수, 간접 명령 수와 Compute Dispatch 수를 기록하고, BRG 경로에서는 BatchDrawCommand 수와 OnPerformCulling 구성 정보를 기록한다. 이러한 값은 동일한 처리 계층의 공통 지표로 간주하지 않고 각 렌더링 경로의 제출 구조를 확인하기 위한 지표로 구분한다. Unity에서 관측되는 실제 Draw 및 Batch 수도 별도의 관측값으로 저장한다.

실행 종료 후에는 평균 FPS, 평균·중앙값 Frame Time, 95·99 백분위 Frame Time 및 1% Low FPS를 산출한다. 인스턴스 수, 가시 인스턴스 수, 활성 렌더러, 제출 방식, GPU 버퍼 용량 및 실행 설정은 측정 결과의 유효성을 확인하기 위한 검증 정보로 함께 기록한다.

측정 중 파일 입출력으로 인한 추가 부하를 줄이기 위해 측정값은 메모리에 우선 수집하고, 실행 종료 후 요약값과 1초 주기 자료를 CSV 파일로 일괄 저장한다.

제2장 설계 상세화 및 변경 내역

2.1 시스템 구조와 실험 조건

착수보고서에서 제시한 ECS 기반 군집 시뮬레이션과 GPU 인스턴스 렌더링 구조는 유지하였다. 현재 시스템은 ECS에서 스웜과 병사의 이동, 대열, 탐지 대상, 교전 및 애니메이션 상태를 갱신하고, 갱신된 위치, 방향, 팩션 식별값 및 애니메이션 상태를 두 렌더링 경로의 공통 원천 데이터로 사용한다.

착수보고서에서 계획한 별도의 공간 Grid 계층은 현재 구현에 포함하지 않았다. 대신 타깃 참조, 유닛 상태 정보와 군체 Footprint를 이용하여 이동 및 교전 상태를 처리한다. 각 렌더러는 동일한 ECS 상태를 읽은 뒤 이를 경로별 인스턴스 데이터 구조로 변환하여 GPU 버퍼에 전달한다(그림 2.1).

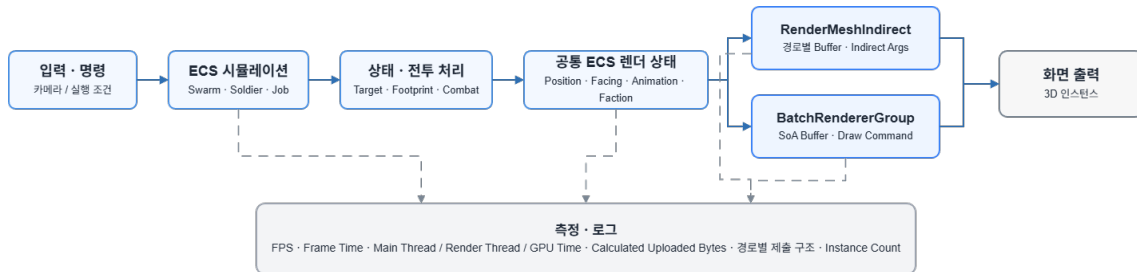


그림 2.1: 현재 프로젝트의 전체 시스템 구조와 측정 계층

비교 조건은 렌더링 경로와 제출 방식을 조합한 네 가지 실행 조건으로 구성한다 (표 2.1). 통합 제출은 각 활성 서브메시의 전체 인스턴스 범위를 하나의 명령으로 구성하며, 4분할 제출은 각 활성 서브메시의 인스턴스 범위를 네 구간으로 나누어 네 개의 명령으로 구성한다.

두 제출 방식은 동일한 인스턴스 집합과 활성 서브메시를 렌더링하며, 명령별 시작 위치와 인스턴스 범위만 다르게 설정한다.

표 2.1: 렌더링 경로와 제출 방식에 따른 실행 조건

조건	렌더링 경로	제출 방식	구현 및 확인 내용
E1	RMI	통합 제출	활성 서브메시별 전체 인스턴스 범위를 하나의 간접 명령과 API 호출로 제출
E2	RMI	4분할 제출	활성 서브메시별 인스턴스 범위를 네 개의 간접 명령과 API 호출로 분할
E3	BRG	통합 제출	동일한 Batch와 Draw Range에 활성 서브메시별 하나의 Batch Draw Command 구성
E4	BRG	4분할 제출	동일한 Batch와 Draw Range에 활성 서브메시별 네 개의 Batch Draw Command 구성

네 조건은 동일한 ECS 시뮬레이션, 초기 배치, 메시, 텍스처, 애니메이션 및 GPU 스키닝 데이터를 사용한다. 두 경로의 인스턴스 데이터 접근 방식이 다르므로 셰이더와 머티리얼은 경로별로 구성하되, 팩션별 색상과 조명 반응을 포함한 시각 파라미터와 렌더 상태는 동등하게 설정한다.

LOD는 0으로 고정하고 Shadow와 Motion Vector는 비활성화한다. 개별 인스턴스에 대한 프러스텀 및 오클루전 컷링은 수행하지 않고, 모든 인스턴스를 가시 대상으로 제출한다.

2.2 구현 구조

착수보고서 이후에도 기존 ECS 군집 시뮬레이션, 병사 상태 데이터, GPU 스키닝 자원과 BRG 렌더러의 기본 구조는 유지하였다. 주요 변경 사항은 동일한 ECS 원천 상태를 사용하는 RMI 렌더러와 렌더링 경로 선택 기능을 추가하고, 두 경로에 동일한 통합, 4분할 제출 조건을 적용한 것이다.

- **공통 ECS 원천 상태.** ECS 시스템과 Job은 위치, 방향, 팩션 식별값 및 애니메이션 상태를 갱신한다. 두 렌더러는 동일한 ECS 상태를 읽지만, 실제 GPU 인스턴스 데이터와 버퍼는 각 렌더링 경로에 맞게 별도로 구성한다. 원거리 공격 상태와 공격 애니메이션은 유지하되, 비교 대상 외의 GameObject 기반 투사체 생성과 시각화는 비활성화한다.
- **렌더링 경로와 제출 구조.** 실행 조건에 따라 RMI 또는 BRG 렌더러만 활성화한다. 두

경로 모두 활성 서브메시별로 통합 제출과 4분할 제출을 지원하며, GPU 버퍼는 실행 중 재할당이 발생하지 않도록 최대 부하 수준을 기준으로 확보한다.

2.2.1 RMI 구현

ECS 상태는 입자、애니메이션 데이터로 변환되며, Compute Shader에서 인스턴스 행렬을 생성한다. CPU는 활성 서브메시별 간접 인자를 구성하여 통합 또는 4분할 방식으로 `Graphics.RMI()`를 호출한다. 세부 데이터 및 명령 흐름은 그림 2.2과 같다.

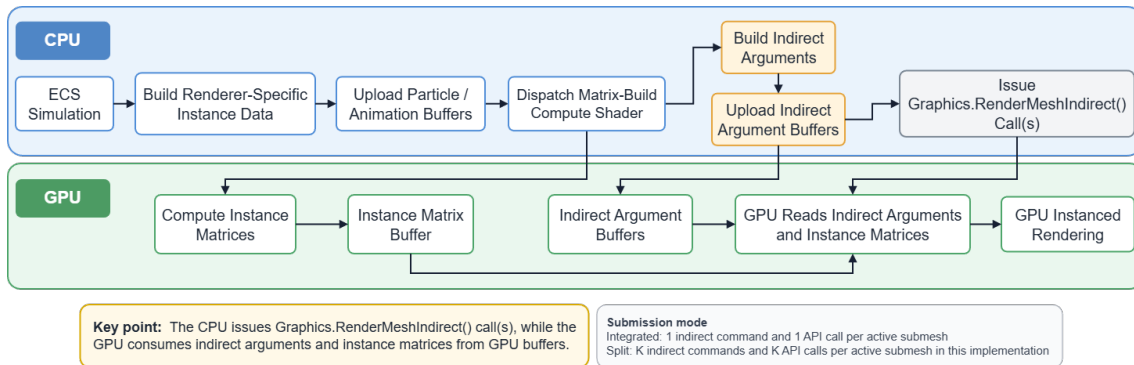


그림 2.2: RMI 렌더러의 실제 구현 구조. ECS 상태를 경로별 데이터로 변환하고 Compute Shader에서 인스턴스 행렬을 생성한 뒤, 활성 서브메시별 간접 명령을 통합 또는 4분할하여 제출한다.

2.2.2 BRG 구현

BRG 렌더러는 동일한 ECS 상태를 위치、방향, 오프셋、크기 및 애니메이션 속성으로 구성된 SoA 형식의 인스턴스 데이터로 변환한다. `OnPerformCulling` 콜백에서는 개별 가시성 판정을 수행하지 않고 전량 가시 목록을 작성하며, 활성 서브메시별 통합 또는 4분할 Batch Draw Command를 구성한다. 세부 흐름은 그림 2.3과 같다.

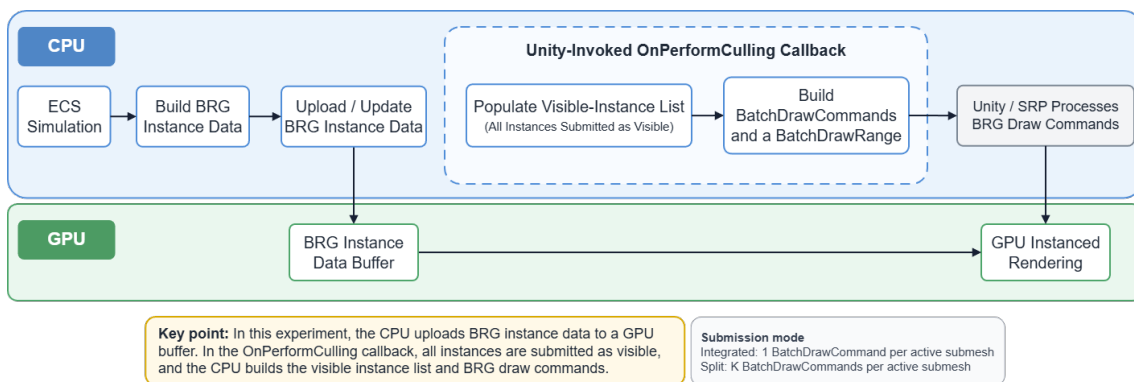


그림 2.3: BRG 렌더러의 실제 구현 구조. ECS 상태를 BRG 인스턴스 버퍼에 기록하고, `OnPerformCulling` 콜백에서 전량 가시 목록과 활성 서브메시별 통합 또는 4분할 Batch Draw Command를 구성한다.

2.3 측정 및 실험 프로토콜

측정 자동화는 다음 네 단계로 구성한다.

- 1) **실행 조건 설정 및 검증** — E1-E4 조건, 총 인스턴스 수, 스왑 분할 수준 및 반복 회차를 적용하고, 활성 렌더러, 셰이더, GPU 버퍼 용량, 인스턴스 수와 예상 명령 수를 확인한다.
- 2) **초기화 및 워밍업** — ECS 엔티티와 렌더링 자원을 초기화한 뒤 3초 동안 워밍업한다. 워밍업 구간의 값은 결과에 포함하지 않는다.
- 3) **측정** — BenchmarkRunner가 실제 경과시간을 기준으로 60초 동안 실행한다. 프레임 처리 시간과 완료 프레임 수는 매 프레임 수집하고, FrameTimingManager와 ProfilerRecorder를 이용한 단계별 시간과 경로별 구조 지표는 1초 주기로 수집한다.
- 4) **집계 및 저장** — 실행 종료 후 요약 지표를 계산하고, 실행 조건, 검증 정보와 측정값을 CSV 파일로 일괄 저장한다. 측정 중에는 파일 기록을 수행하지 않고 값을 메모리에 우선 보관한다.

각 실행은 동일한 씬, 해상도, 카메라, 초기 배치와 난수 시드를 사용한다. VSync와 목표 Frame Rate 제한은 해제하며, 설정 검증에 실패하거나 측정값이 누락된 실행은 동일한 조건으로 다시 수행한다. 부하 조건과 측정 항목은 표 2.2, 표 2.3과 같다.

표 2.2: 실행 부하 조건

항목	수준 / 값
총 인스턴스 수 N	2,000, 4,000, 8,000, 16,000
스왑 분할 수준 L	4, 8, 16
부하 조합	4수준 $\times$ 3수준 = 12개
워밍업 시간	실행당 3초
측정 시간	실행당 60초
조건당 반복	5회
총 실행 횟수	12개 부하 조합 $\times$ 4조건 $\times$ 5회 = 240회

총 인스턴스 수는 활성 스왑에 균등하게 배치한다. 모든 N 은 L 과 분할 수 4로 나누어떨어지도록 설정하여 스왑별 인스턴스 수 N/L 과 4분할 조건의 명령별 인스턴스 범위 $N/4$ 를 균등하게 유지한다.

표 2.3: 측정 및 검증 항목

항목	수집 및 확인 내용
실제 경과시간 기반 Frame Time	매 프레임 처리 시간 기록 및 프레임 분포 산출
완료 프레임 수, 평균 FPS	60초 측정 구간의 전체 실행 성능 요약
평균, 중앙값, 95/99 percentile	실행별 Frame Time 요약값 산출
1% Low FPS	처리 시간이 긴 프레임 구간의 상태 확인
Main Thread Time	ECS 시뮬레이션, 렌더 데이터 구성과 기타 작업을 포함한 전체 Main Thread 처리 시간
Render Thread Time	Unity Render Thread의 전체 처리 시간
GPU Time	해당 프레임의 전체 GPU 처리 시간
Logical Submitted Commands	활성 서브메시 수와 제출 분할에 따라 구성된 논리적 명령 수
Calculated Uploaded Bytes	렌더러가 명시적으로 갱신한 인스턴스 및 명령 데이터의 계산상 바이트 수
RMI 경로 지표	API 호출 수, 간접 명령 수 및 Compute Dispatch 수
BRG 경로 지표	Batch Draw Command 수와 OnPerformCulling 구성 정보
Observed Draw, Batch Count	Unity에서 관측된 실제 Draw 및 Batch 수
Instance, Visible Instance Count	설정 인스턴스 수와 전량 가시 제출 여부 확인

Main Thread Time과 Render Thread Time은 특정 렌더러 함수만을 분리한 값이 아니라, 해당 프레임에서 각 스레드가 수행한 전체 처리 시간이다. 또한 RMI의 API 호출 및 간접 명령, BRG의 Batch Draw Command, Unity에서 관측한 실제 Draw는 서로 다른 처리 계층의 값이므로 동일한 단위로 직접 대응하지 않는다.

Calculated Uploaded Bytes는 하드웨어에서 직접 측정한 전체 CPU-GPU 전송량이 아니며, 각 렌더러가 명시적으로 갱신한 인스턴스와 명령 버퍼의 크기를 기준으로 계산한 값이다.

2.4 기대 결과 및 해석

중간 단계에서는 특정 렌더링 경로의 성능 우위를 전제하지 않고, 네 실행 조건과 측정 도구가 설계한 구조대로 동작하는지를 우선 확인한다. 두 렌더링 경로에서 동일한 ECS 상태, 인스턴스 수, 메시와 애니메이션이 사용되어야 하며, 활성 서브메시별 통합 및 4분할 명령 수가 설정과 일치해야 한다.

또한 12개 부하 조합과 E1-E4 조건에 대해 워밍업, 측정, 반복 회차 관리와 CSV 저장이 자동으로 수행되어야 한다. 중간보고서에는 렌더러와 계측 도구의 구현 상태 및 동작 확인 결과를 제시하고, 조건별 성능 비교와 단계별 비용 해석은 본 측정 완료 후 최종보고서에 반영한다.

2.5 착수보고서 대비 변경점

표 2.4: 설계 측면의 주요 변경점

항목	착수보고서	현재(수정)
비교 조건	Render-only BRG, ECS-fed BRG 및 BRG 내부 제출 조건	RMI/BRG와 통합/4분할 제출을 조합한 E1-E4 조건
ECS 적용	Render-only와 ECS-fed 조건을 구분	모든 조건에서 동일한 ECS 시뮬레이션 상태 사용
상태 처리	별도 공간 Grid와 질의 계층 계획	타깃 참조와 군체 Footprint 기반 상태, 교전 처리
렌더링 구현	BRG 중심의 렌더링 제출 구조	기존 BRG와 GPU 스키닝 구조를 유지하고 RMI 렌더러 추가
렌더 데이터	BRG용 인스턴스 데이터 중심	동일한 ECS 원천 상태를 각 렌더러가 경로별 GPU 데이터로 변환
제출 단위	유형별 분할 및 통합 Draw List	활성 서브메시별 인스턴스 범위의 통합 및 4분할 제출
부하 및 측정	인스턴스 수 200~10,000, 논리 유닛 수 2, 4, 8, 총 300회 계획	인스턴스 수 2,000~16,000, 스웬 분할 수준 4, 8, 16, 3초 워밍업과 60초 측정, 총 240회 실행
측정 구조	FPS, Frame Time, Draw Command와 Visible Count 중심	프레임별 성능값과 1초 주기 지표를 구분하고, 계산상 버퍼 갱신량과 경로별 제출 정보를 자동 저장

제3장 추진 계획 및 진행 현황

본 프로젝트는 학과에서 제시한 졸업과제 공식 일정을 기준으로 추진한다. **한국멀티미디어학회 논문 제출(2026. 7. 30)**은 프로젝트 구현 및 측정 결과를 활용하는 병행 마일스톤으로 관리하며, 중간, 최종보고서 제출과 발표심사 등 졸업과제 공식 산출물의 완성을 우선한다.

착수보고서 이후 연구 주제와 비교 조건은 변경되었으나, 기존 ECS 군집 시뮬레이션, GPU 스키닝 자원 및 BRG 렌더러를 대부분 유지할 수 있어 전체 추진 순서는 크게 변경되지 않았다. 현재는 RMI 경로와 통합, 4분할 제출 조건을 추가하고, 네 실행 조건의 빌드 검증과 자동 측정 기능을 보완하는 단계이다. 이후 240회 본 측정, 결과 정리, 최종보고서 및 발표 자료 작성 순으로 진행한다. 세부 진행 현황은 표 3.1와 같다.

표 3.1: 추진 계획 및 진행 현황 (2026년, 월 단위)

분류	세부 내용	3	4	5	6	7	8	9	10
졸업 과제	착수보고서, 지도확인서 제출 (~5.16)								
	구현, 중간보고서 제출 (~7.18)								
	최종보고서, 최종평가표 제출 (~9.19)								
	발표심사 및 결과물 정리 (9.30/10.01)								
시뮬 레이션	ECS 군집 시뮬레이션 기반 병사 상태, 전투, 애니메이션 부하 조건, 실행 씬 구성								
	공통 GPU 스키닝, BRG 기반								
	RMI 통합, 4분할 (E1, E2)								
	BRG 통합, 4분할 (E3, E4)								
렌더링 경로	네 조건 빌드, 설정 검증								
	측정 자동화 도구 빌드 실행, 1초 주기 계측 검증								
	240회 본 측정, 결과 정리								
	중간보고서, 기술 문서 논문 초고, 학회 제출 (7.30)								
실험, 측정	최종보고서, 발표 자료, 결과물								
문서화, 제출									

■ 완료

■ 진행

■ 예정

▲ 마일스톤(7.30 학회 제출)

3.1 구성원별 진척도

구성원별 담당 영역은 ECS 시뮬레이션과 렌더링 구조, 애니메이션 및 인스턴스 유형, 성능 측정 자동화와 실험 분석으로 구분하였다. 구성원별 작업은 기능 단위로 구분하되, 렌더링 경로와 애니메이션의 연동, 측정 항목의 유효성 및 결과 해석은 내용을 공유하고 공동으로 검토한다. 각 구성원은 담당 기능의 핵심 구현을 완료하거나 검증 단계까지 진행하였으며, 현재는 네 실행 조건의 통합 검증과 본 측정을 위한 보완 작업을 공동으로 수행하고 있다.

3.2 구성원별 진척도

구성원별 담당 영역은 ECS 군집 시뮬레이션과 렌더링 구조, 군집 애니메이션과 인스턴스 유형 구성, 성능 측정 자동화와 실험 분석으로 구분하였다. 각 구성원은 담당 기능의 핵심 구현을 수행하였으며, 현재는 네 실행 조건의 통합 검증과 본 측정을 위한 보완 작업을 진행하고 있다.

3.2.1 김재식: ECS 시뮬레이션 및 렌더링 구조

ECS 기반 논리 유닛과 병사 인스턴스의 데이터 구조를 설계하고, 군집 이동, 대열 유지, 방향 전환, 탐지, 교전 및 병사별 상태 갱신을 수행하는 시뮬레이션 기반을 구현하였다. 또한 ECS에서 갱신된 위치, 방향, 팩션 및 애니메이션 상태를 GPU 렌더링 경로에 전달하는 구조를 구성하였다. 기존 BRG 렌더러와 GPU 스키닝 구조를 유지하면서 동일한 ECS 상태를 사용하는 `RenderMeshIndirect` 렌더러를 추가하였다. 두 렌더링 경로에는 활성 서브메시별 통합 제출과 4분할 제출 기능을 적용하여 E1-E4 실행 조건의 기반을 구현하였다.

3.2.2 이천서: 군집 애니메이션 및 인스턴스 유형 구성

GPU 스키닝 기반 휴머노이드의 애니메이션 상태와 모션 데이터를 구성하고, ECS에서 갱신되는 이동 및 전투 상태가 인스턴스별 애니메이션으로 연결되도록 구현하였다. 다수의 인스턴스가 동일한 동작을 일률적으로 재생하지 않도록 애니메이션 위상과 재생 상태를 분산하는 기능도 적용하였다. 근접 및 원거리 유닛과 팩션별 시각 구분 등 인스턴스 유형을 확장하고, 스위치 생성과 유형별 배치를 자동화하였다. 이를 바탕으로 이동, 대기, 공격 등 상태 변화에 따라 다수 인스턴스의 애니메이션이 전환되는 것을 확인하였다.

3.2.3 안지홍: 휴머노이드 렌더링과 성능 측정 자동화 및 실험 분석

FPS, Frame Time, Main Thread Time, Render Thread Time 및 GPU Time을 수집하기 위한 측정 도구를 구현하였다. 측정 도구는 실행 조건 적용, 워밍업, 측정 구간 관리, 프레임별 자료 수집, 1초 주기 계측, 실행별 통계 산출 및 CSV 결과 저장 기능으로 구성하였다. RMI의 API 호출 수, 간접 명령 수와 Compute Dispatch 수, BRG의 BatchDrawCommand 수 등 경로별 제출 구조 지표를 기록하는 기능도 추가하였다. 이와 함께 휴머노이드 프리팹과 3D 에셋을 정리하고, 애니메이션 GPU 스키닝 결과가 출력되도록 연결하였다.

3.3 위험 대응

연구 주제와 비교 조건의 변경으로 일부 렌더링 경로와 계측 기능의 구현 일정이 조정되었으나, 기존 ECS 시뮬레이션, GPU 스키닝 및 BRG 구현을 유지하여 전체 일정에 미치는 영향을 최소화하였다. 이후에는 기능 추가 범위를 확정하고, 렌더링 경로 검증, 예비 측정, 본 측정, 결과 정리의 순서로 작업 단계를 구분하여 각 단계의 완료 여부를 주 단위로 점검한다.

총 240회의 반복 측정은 장시간이 소요되고 실행 오류에 따른 재측정 가능성이 있으므로, 본 측정 일정과 별도로 예비 측정 및 재측정 기간을 확보한다. 측정이 지연되는 경우에는 구현 범위를 추가로 확대하지 않고 검증이 완료된 조건부터 순차적으로 실행하며, 측정과 동시에 완료된 결과의 정리 및 문서화를 병행한다. 이후 실험 결과를 반영하여 최적화 데모 씬을 완성하는 것을 목표로 한다.

제4장 중간 결과

현재까지 ECS 군집 시뮬레이션과 GPU 스키닝 애니메이션, 두 렌더링 경로의 통합, 4분할 제출 기능 및 성능 측정 도구를 구현하였다. 각 기능은 동일한 프로젝트와 실행 실행에서 연동되며, 실행 조건에 따라 렌더링 경로와 제출 방식을 전환할 수 있도록 구성하였다.

4.1 구현 데모

기존 ECS 군집 시뮬레이션에서는 스웩 이동, 대열 유지, 탐지, 교전 및 병사별 애니메이션이 정상적으로 동작한다. 동일한 시뮬레이션 상태를 사용하는 RMI와 BRG 경로에서도 인스턴스의 위치, 방향, 팩션별 시각 구분 및 GPU 스키닝 애니메이션이 동일하게 출력되는 것을 확인하였다.

E1-E4 조건을 전환하면 선택한 렌더러만 활성화되고, 통합 제출과 4분할 제출에 따라 활성 서브메시별 논리적 명령 수가 변경된다. 두 제출 방식에서 전체 인스턴스 수와 화면 출력 결과는 동일하게 유지되며, GameObject 기반 투사체 시각화와 개별 인스턴스 컬링 등 비교 조건 외 기능은 성능 평가를 위해 비활성화 기능을 지원한다.

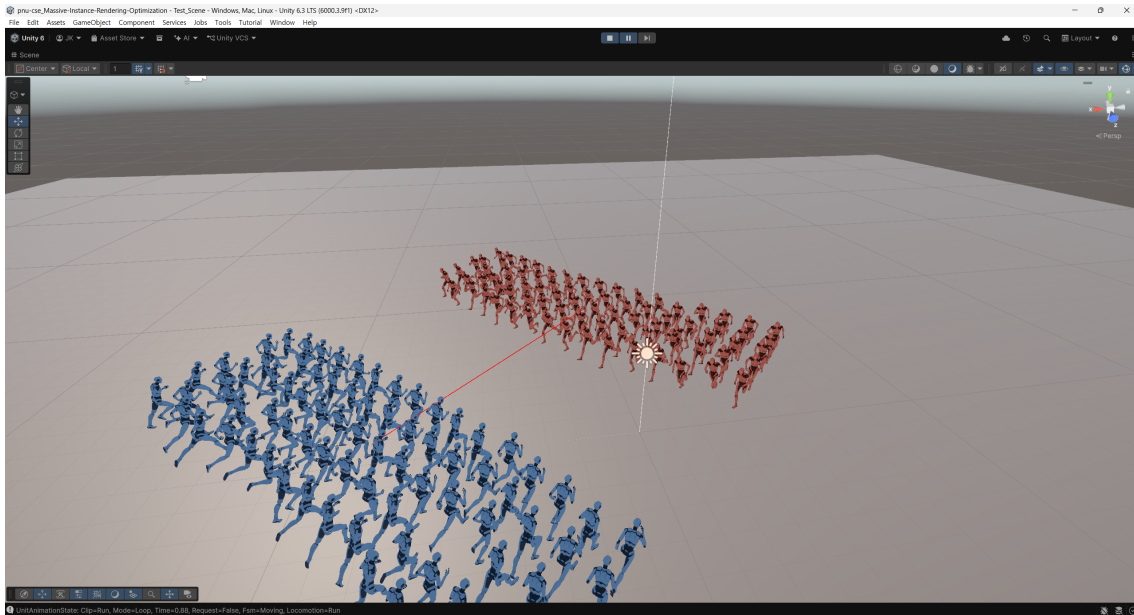


그림 4.1: ECS 군집 시뮬레이션을 적용한 렌더링 실행 화면

4.2 실험, 측정 도구

측정 도구에서는 E1-E4 조건, 총 인스턴스 수 N , 스왑 분할 수준 L 및 반복 회차를 설정할 수 있다. 시험 실행을 통해 설정한 렌더링 경로와 제출 방식이 적용되고, 인스턴스 수와 논리적 명령 수 등 실행 검증 정보가 조건에 맞게 갱신되는 것을 확인하였다.

프레임 처리 시간과 완료 프레임 수는 매 프레임 수집되며, Main Thread Time, Render Thread Time, GPU Time과 경로별 구조 지표는 실제 경과시간 기준 1초 주기로 기록된다. RMI 경로에서는 API 호출 수, 간접 명령 수와 Compute Dispatch 수가 기록되고, BRG 경로에서는 Batch Draw Command 수가 기록된다.

측정 종료 후에는 평균 FPS, 평균, 중앙값 Frame Time, 95, 99 percentile Frame Time, 1% Low FPS와 실행 조건별 측정값이 집계된다. 시험 실행 결과 수집한 자료와 실행 설정이 CSV 파일로 출력되는 것을 확인하였다.

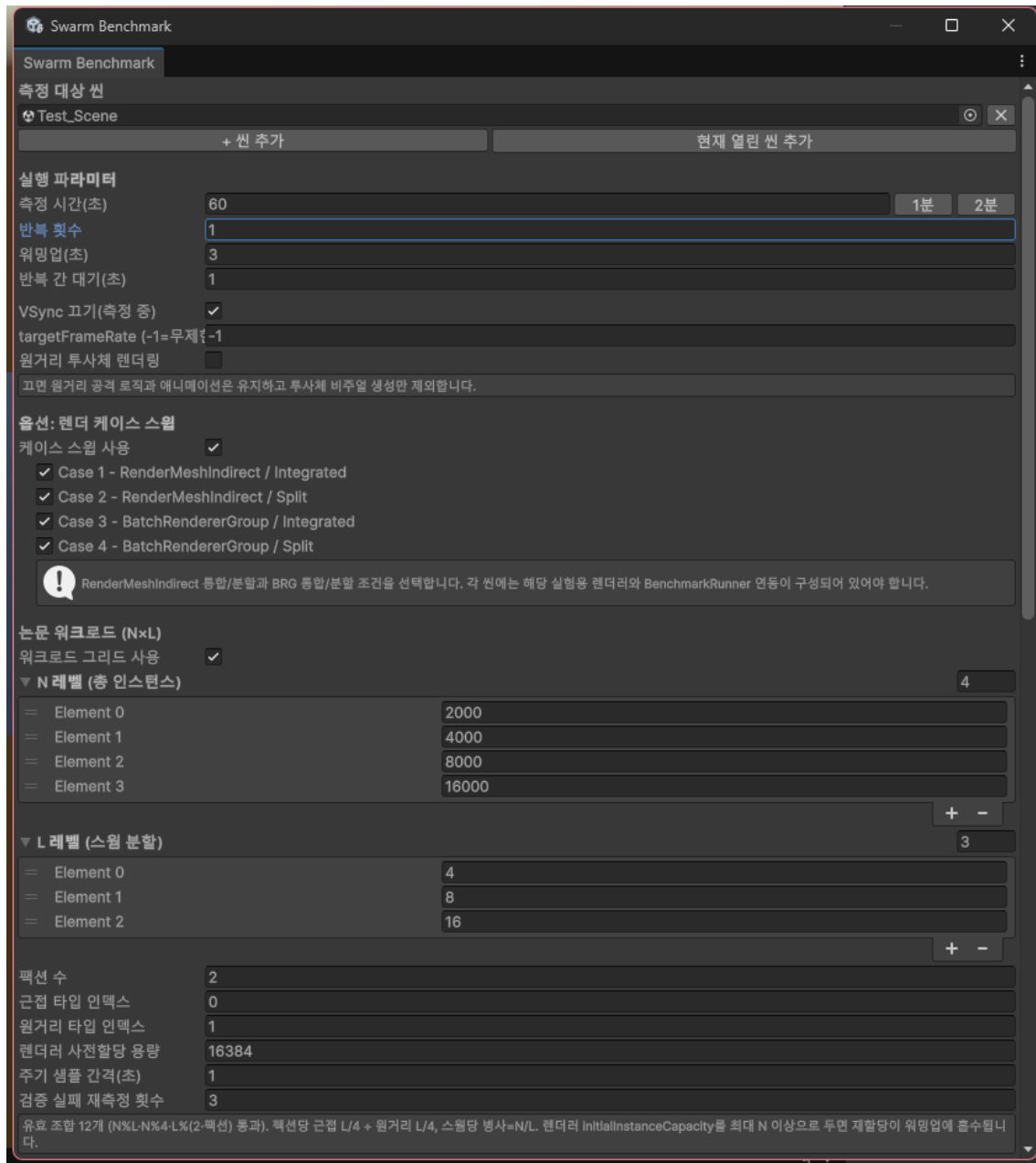


그림 4.2: 실행 조건 설정, 성능 지표 수집용으로 구현한 툴

현재 구현 결과를 통해 동일한 ECS 시뮬레이션에서 네 실행 조건을 전환하고, 렌더링 결과와 제출 구조를 확인하며, 조건별 성능 자료를 자동으로 수집, 저장할 수 있는 실험 실행 기반을 확보하였다.

참고 문헌 및 자료

- [1] A. Beacco, N. Pelechano, and C. Andújar, “A Survey of Real-Time Crowd Rendering,” *Computer Graphics Forum*, vol. 35, no. 8, pp. 32–50, 2016.
- [2] Y. Dong and C. Peng, “Real-Time Large Crowd Rendering with Efficient Character and Instance Management on GPU,” *International Journal of Computer Games Technology*, vol. 2019, Article ID 1792304, pp. 1–15, 2019.
- [3] D. Wingqvist, F. Wickström, and S. Memeti, “Evaluating the Performance of Object-Oriented and Data-Oriented Design with Multi-Threading in Game Development,” *Proc. IEEE Games, Entertainment, Media Conference (GEM)*, pp. 1–6, 2022.
- [4] L. Voisard, H. D. F. Serra, C. Politowski, F. Petrillo, and Y.-G. Guéhéneuc, “A Mapping Study of the Entity Component System Pattern,” *Proc. IEEE/ACM 9th Int. Workshop on Games and Software Engineering (GAS)*, pp. 33–40, 2025.
- [5] M. K. Sung, “Performance Comparison Analysis of Entity Component System Frameworks for Game Development,” *Journal of Korea Multimedia Society*, vol. 28, no. 1, pp. 58–65, 2025.
- [6] M. Turpeinen, “A Performance Comparison for 3D Crowd Rendering using an Object-Oriented System and Unity DOTS with GPU Instancing on Mobile Devices,” Master’s Thesis, KTH Royal Institute of Technology, 2020.
- [7] Unity Technologies, “BatchRendererGroup API in URP,” Unity Manual / Entities Graphics Manual, Unity 6.3 / Entities Graphics 1.4, 2026. <https://docs.unity3d.com/6000.3/Documentation/Manual.html>
- [8] S. Liu, M. Claypool, A. Kuwahara, and J. J. Scovell, “The Effects of Frame Rate Variation on Game Player Quality of Experience,” *Proc. CHI Conference on Human Factors in Computing Systems*, Article 573, pp. 1–10, 2023.
- [9] Unity Technologies, “Unity 6 Documentation (ver. 6000.3),” Unity, 2025. <https://docs.unity3d.com/6000.3/Documentation/Manual/index.html>